C++ PROGRAMMING ASSIGNMENT – 02

Hard-Level Logic Building & Fundamental Programming

Course	Programming in C++	**Difficulty**	Hard
Assignment	02	**Total Questions**	10
Topics	Conditions, loops, functions, strings, basic arrays		

GENERAL INSTRUCTIONS

- Write a separate, well-structured C++ program for each question.
- Use only fundamental C++ constructs: variables, operators, if-else, switch, loops, user-defined functions, strings and basic arrays.
- Do not use STL containers, sorting/searching algorithms, recursion, pointers or other DSA techniques.
- Use functions to divide a program into logical tasks wherever appropriate.
- Validate inputs and handle boundary cases explicitly.
- The sample input/output is illustrative; your program should work for other valid inputs too.

SECTION A – CONDITIONAL STATEMENTS & LOOPS

Question 01 – Bank ATM Transaction Simulator

Write a menu-driven program that simulates a basic ATM. The initial account balance is Rs. 10,000. The user can perform multiple transactions until choosing Exit.

Requirements

- Display menu options: Check Balance, Deposit, Withdraw, Change PIN and Exit.
- A withdrawal is allowed only when the amount is positive, does not exceed the balance, and is a multiple of Rs. 100.
- A deposit must be positive. Reject invalid amounts without changing the balance.
- For Change PIN, ask for the current 4-digit PIN before accepting a new 4-digit PIN.
- Allow at most three incorrect PIN attempts for a withdrawal; lock further withdrawals for the current program run after three failures.
- Use a loop for the menu and switch for menu selection. Keep the program running until Exit.

Sample Input

```
Select option: 1
Select option: 3
Enter amount: 2500
```

Sample Output

```
Withdrawal successful.
Remaining balance: Rs. 7500
```

Question 02 – Time and Date Calculator

Write a program that accepts two valid times in 24-hour format (hours, minutes and seconds) and calculates the elapsed time from the first time to the second time. The second time may fall on the following day.

Requirements

- Validate both times: hours 0–23, minutes 0–59 and seconds 0–59.
- Convert each time into total seconds from midnight.
- If the second time is earlier than the first, treat it as the next day.
- Display the elapsed duration in hours, minutes and seconds.
- Also accept a date (day, month, year) and validate it using leap-year rules before performing the time calculation.

Sample Input

```
Date: 28 2 2024
Start time: 23 45 30
End time: 1 15 10
```

Sample Output

```
Elapsed time: 1 hour 29 minutes 40 seconds
```

SECTION B – NUMBER LOGIC & NESTED LOOPS

Question 03 – Range-Based Number Property Report

Accept two positive integers L and R (where L is not greater than R). Print a report of all numbers in the inclusive range that satisfy at least one of the following properties: palindrome, Armstrong number, or Strong number.

Requirements

- Create separate functions for checking palindrome, Armstrong and Strong numbers.
- For Armstrong checking, determine the number of digits and raise each digit to that count using loops (do not use pow()).
- For Strong checking, calculate digit factorials using a loop.
- Print each qualifying number followed by the property or properties it satisfies.
- At the end, display the count of palindrome, Armstrong and Strong numbers found in the range.
- Do not convert numbers to strings.

Sample Input

```
Input:
L = 1
R = 500
```

Sample Output

```
Illustrative output:
1 : Palindrome, Armstrong, Strong
2 : Palindrome, Armstrong, Strong
...
153 : Palindrome? No, Armstrong
145 : Strong
Counts are displayed at the end.
```

Question 04 – Digit-Level Number Transformation

Accept a non-negative integer N and construct a new number by replacing each digit according to the following rules: even digits become the next odd digit, odd digits become the previous even digit, and digit 9 becomes 0.

Requirements

- Process the digits using arithmetic operations only; do not convert the number into a string.
- Preserve the original digit positions while constructing the transformed number.
- Display the original number, transformed number, sum of transformed digits and number of zeroes in the transformed number.
- Handle N = 0 correctly.
- If the transformed number begins with zero, display its numeric value (leading zeroes are not retained).

Sample Input

```
Input:
N = 583209
```

Sample Output

```
Transformed number: 472108
Sum of transformed digits: 22
Number of zeroes: 1
```

Question 05 – Hollow Diamond Pattern Generator

Accept an odd positive integer N representing the total number of rows of a hollow diamond. Print a hollow diamond using asterisks.

Requirements

- Use nested loops only; do not use string repetition or pattern libraries.
- The first and last rows must contain one asterisk.
- The middle row must contain N asterisks.
- All interior positions must be spaces.
- Reject even values and values less than 1.

Sample Input

```
Input:
N = 7
```

Sample Output

```
Output:
*
* *
* *
* *
* *
* *
*
```

SECTION C – FUNCTIONS & MENU-DRIVEN PROGRAMMING

Question 06 – Student Result and Grade Management

Write a program that accepts marks of a student in five subjects and generates a detailed result report.

Requirements

- Create a function to validate each mark (valid range: 0 to 100). If any mark is invalid, ask for that mark again.
- Create functions to calculate total marks, percentage and grade.
- A student passes only if every subject has at least 35 marks.
- If the student fails in any subject, the overall result must be FAIL regardless of the percentage.
- For passed students, assign grades: A for 90% and above, B for 75–89.99%, C for 60–74.99%, D for 45–59.99%, and E otherwise.
- Display subject-wise marks, total, percentage, result and grade.

Sample Input

```
Input:
Marks: 92 81 76 64 87
```

Sample Output

```
Output:
Total: 400 / 500
Percentage: 80%
Result: PASS
Grade: B
```

Question 07 – Recursive-Free Mathematical Menu

Create a menu-driven program that repeatedly accepts a choice and performs one of the following operations on a positive integer: factorial, sum of digits, reverse, prime check, GCD of two numbers, or exit.

Requirements

- Implement every operation using a separate user-defined function.
- Do not use recursion or built-in GCD functions.
- Calculate factorial using a loop and detect when the result would exceed the range of the chosen integer type.
- For GCD, use repeated remainder operations (Euclidean method).
- Validate inputs and handle invalid menu choices.
- The menu must continue until the user selects Exit.

Sample Input

```
Input:
Choice: 5
First number: 84
Second number: 30
```

Sample Output

```
GCD: 6
```

SECTION D – STRINGS & BASIC ARRAYS

Question 08 – Sentence Analysis and Word Formatting

Accept a complete line of text and produce a detailed analysis of the sentence.

Requirements

- Count uppercase letters, lowercase letters, digits, spaces and special characters.
- Count words while treating any sequence of spaces as a single word separator.
- Find and display the longest word. If multiple words share the maximum length, display the first one.
- Create a second output sentence in which the first letter of every word is uppercase and all remaining letters are lowercase.
- Do not use stringstream, regular expressions or built-in case-conversion functions.
- Preserve punctuation in the formatted sentence.

Sample Input

```
Input:
c++ IS a GREAT language 123!
```

Sample Output

```
Words: 5
Uppercase letters: 7
Lowercase letters: 9
Digits: 3
Longest word: language
Formatted sentence: C++ Is A Great Language 123!
```

Question 09 – Array-Based Class Performance Report

Accept marks of N students in three subjects and generate a class performance report. Use a two-dimensional array or three separate one-dimensional arrays.

Requirements

- Validate every mark to be within 0–100.
- For each student, calculate total, average and pass/fail status. A student passes only if every subject mark is at least 35.
- Display the class average across all students and subjects.
- Count the number of students who passed and failed.
- Find the highest total and display the student number(s) who achieved it.
- Do not sort the data or use STL containers.

Sample Input

```
Input:
Number of students: 3
Student 1: 80 70 90
Student 2: 30 85 75
Student 3: 90 90 90
```

Sample Output

```
Student 1: Total 240, Average 80, PASS
Student 2: Total 190, Average 63.33, FAIL
Student 3: Total 270, Average 90, PASS
Class average: 77.78
Passed: 2
Failed: 1
Highest total: 270 (Student 3)
```

Question 10 – Array-Based Daily Temperature Report

A weather station records the temperature for N consecutive days. Accept the temperatures into an array and generate a report.

Requirements

- Display the highest and lowest temperatures and the day number on which each first occurred.
- Calculate the average temperature.
- Count how many days had temperatures above the average, below the average and exactly equal to the average.
- Count the number of times the temperature increased compared with the previous day.
- Display the longest consecutive run of days during which the temperature was strictly above the average.
- If multiple runs have the same maximum length, report the first run.
- Use basic array traversal and loops only; do not sort the array.

Sample Input

```
Input:
N = 7
Temperatures: 28 30 32 31 34 35 29
```

Sample Output

```
Highest: 35 (Day 6)
Lowest: 28 (Day 1)
Average: 31.29
Above average: 4 days
Below average: 3 days
Equal to average: 0 days
Temperature increases: 4
Longest above-average run: 4 days (Day 3 to Day 6)
```

END OF ASSIGNMENT

Submission checklist

- Submit ten separate, compilable C++ source files.
- Mention the assignment and question number in each file.
- Include meaningful variable names, indentation and brief comments.
- Test normal cases, boundary cases and invalid inputs.
- Be prepared to explain the logic and control flow used in each program.

Learning objective: Strengthen independent problem-solving through layered conditions, nested loops, modular functions, character processing and basic array traversal, without relying on DSA or advanced library features.